Trabajo práctico 2: revisión del proyecto

Lógica, pasos, resultados y decisiones

Alejandro Quesada Jose Herrera Manfred Azofeifa

26 de octubre de 2026

Resumen

Revisión interna de los puntos 1 y 2: regresión logística propia, PCA y detección por histogramas con MPNet de 768 características. Se verificaron 30 corridas por método. La línea base de 3.1 se ejecutó en la RTX 3080 Ti: Qwen2.5-3B-Instruct obtuvo F1 0.5452 ± 0.0090 en 30 muestras. `pytest` y la celda del notebook dan 58 pruebas superadas. La auditoría corrigió la derivada de la sigmoide en cero y comparó alternativas de calibración. Ampliar la regresión no mejoró F1; calibrar el detector redujo falsas alarmas, pero dejó escapar más spam. La variante $D = 2$ del equipo difiere de reutilizar $D = 384$, como pide la lectura literal del enunciado: su aceptación sigue pendiente. Se mantienen las configuraciones vigentes. Las falsas alarmas limitan la precisión de la línea base LLM. Tras cerrar 3.1, la persona autoriza respaldar sus cambios en main y completar 3.2. Mistral 7B NF4 obtiene F1 0.5518 y Qwen 14B NF4, 0.7126 en las mismas muestras. En 3.3, una época de LoRA sobre Qwen 14B NF4 sube el F1 de 0.7120 a 0.9708 en una prueba reservada fija (un solo entrenamiento), sobre todo al eliminar falsas alarmas.

Índice

1. Alcance y organización **3**

2. Auditoría del dataset y sus dimensiones **3**

3. Puntos 1.1 y 1.2: modelo básico con sesgo **4**

4. Punto 1.3: demostración aislada **4**

5. Punto 1.4: extracción y evaluación corregidas **4**
5.1. Validación dentro de cada corrida 4
5.2. Resultados de la nueva ejecución 5

6. Punto 1.5: reducción y comparación con PCA **6**
6.1. Resultados y dimensión óptima 6

7. Punto 2.1: densidades por histograma y umbral **7**

8. Punto 2.2: evaluación en 30 corridas **9**

9. Auditoría de los puntos 1 y 2 (24/10/2026) **10**
9.1. Cobertura del enunciado 10
9.2. Límites de las conclusiones anteriores 11
9.3. Experimento adicional reproducible 11

10.Punto 3.1: clasificación con un modelo de lenguaje (26/10/2026) **12**
10.1. Decisiones . . . 12
10.2. Lógica y pasos . . . 12
10.3. Pruebas y verificación . . . 13
10.4. Resultado de la línea base . . . 13
10.5. Cierre de verificación . . . 13

11.Punto 3.2: modelos adicionales cuantizados (26/10/2026) **14**
11.1. Resultados y verificación de 3.2 . . . 14
11.2. Conexión local y recuperación de la carga . . . 15

12.Punto 3.3: ajuste eficiente **15**
12.1. Ejecución y resultados de 3.3 . . . 15

13.Puntos pendientes **16**

14.Continuación en main y kernel GPU remoto (26/10/2026) **16**

15.Registro de decisiones **18**

16.Verificación y reproducción **21**

1. Alcance y organización

El enunciado pide detectar spam de SMSSpamCollection (UCI) mediante regresión logística propia, detección de anomalías por histogramas y modelos grandes de lenguaje. La entrega indicada en el enunciado es el 20 de octubre de 2026.

Desde el 24 de octubre se concentra toda la implementación en `notebooks/tp2_detector_spam.ipynb`: utilidades, funciones de los tres puntos, experimentos y análisis. Se retira la regla de un archivo por punto y el paquete `src/tp2/`. Se conserva la organización por apartados dentro del cuaderno, sin copias de las funciones en archivos Python.

Ubicación	**Responsabilidad**
Cuaderno principal	Definiciones, configuración, ejecución y análisis en celdas diferenciadas.
`tests/`	Pruebas externas que ejecutan las definiciones del cuaderno.
`docs/report/sections/`	Informe LaTeX, organizado por punto.
`results/`	Figuras y tablas producidas por el cuaderno.
`data/raw/`	Dataset original versionado, sin modificaciones.
`data/processed/`	Cachés regenerables, excluidas de Git.

Las celdas reutilizables comienzan por `# Definiciones:`. `tests/notebook_loader.py` las ejecuta en orden en un espacio de nombres común, sin ejecutar descargas, configuración de exportación ni experimentos. Así las pruebas comprueban la implementación que se entrega. Las dependencias opcionales se importan dentro de las funciones que las usan.

Se mantienen los contratos: características `float32` de forma (n, D); etiquetas y predicciones `int64` de forma $(n,)$; ham $= 0$, spam $= 1$ y spam como clase positiva del F1. Las funciones reciben datos y parámetros explícitamente y llevan documentación Doxygen en español.

2. Auditoría del dataset y sus dimensiones

El archivo local `data/raw/SMSSpamCollection` coincide byte por byte con el del ZIP oficial de UCI. Su SHA256 es `7d039a24a6083ed9ef0f806ebad56bbb976e3aeb8de05669173bfdc4996c239d`. Tiene 5574 filas: 4827 ham y 747 spam. Cada fila contiene etiqueta y texto; no contiene 768 columnas de características. No hay mensajes vacíos ni errores de UTF-8. Hay 5171 textos únicos y 403 repeticiones adicionales, sin etiquetas contradictorias para un mismo texto.

De dónde sale D. El codificador anterior, `sentence-transformers/all-MiniLM-L6-v2`, produce 384 características. El equipo confirmó cambiar a `sentence-transformers/all-mpnet-base-v2`, que produce 768. Las fichas oficiales describen ambas dimensiones: MiniLM y MPNet. La dimensión depende del codificador; cambiarla exige recalcular características, calibración, métricas, tablas y figuras.

Limitación de duplicados. Se conserva el archivo original y el protocolo estratificado por filas. Por ello un texto idéntico puede aparecer en entrenamiento y prueba, lo que limita la interpretación como generalización a mensajes nuevos. Una evaluación agrupada por texto sería otro protocolo y requiere acordarla con el equipo; no se deduplicó silenciosamente.

3. Puntos 1.1 y 1.2: modelo básico con sesgo

El enunciado pide regresión logística. La confirmación del equipo mantiene una parte lineal básica con sesgo y una sigmoide:

$$z = Xw + b, \qquad p(y = 1 \mid x) = \sigma(z).$$

`forward` calcula probabilidades; el sesgo es un escalar entrenable. La sigmoide se evalúa con $e^{-|z|}$ para evitar desbordamientos con logits extremos. El entrenamiento propio minimiza la entropía cruzada binaria media con descenso del gradiente por lotes completos:

$$\nabla_w L = X^\top (p - y)/n, \qquad \partial L/\partial b = \overline{p - y}.$$

Se actualizan tanto w como b. No se utilizan `torch.nn`, optimizadores de `torch.optim` ni clasificadores ya implementados. La pérdida se calcula a partir de logits de forma estable. El historial conserva la pérdida inicial y una por época.

Las cuatro pruebas obligatorias verifican probabilidades con pesos nulos, sigmoide y estabilidad extrema, primer paso de gradiente (incluido el sesgo) y clasificación reproducible de datos separables. En pruebas se permite Autograd como referencia independiente.

4. Punto 1.3: demostración aislada

El conjunto sintético bidimensional verifica el entrenamiento y la evolución de su frontera frente a una recta conocida con sesgo. Se conservan sus propias semillas, tasa y épocas. Sus datos, modelo e hiperparámetros no se usan en 1.4, 1.5 ni 2: esos apartados trabajan exclusivamente con los mensajes UCI y sus características. Las funciones genéricas `forward` y de entrenamiento son compartidas; los resultados experimentales no lo son.

5. Punto 1.4: extracción y evaluación corregidas

Se codifica cada mensaje original con MPNet y se verifica forma $(5574, 768)$, tipo `float32`, orden de las filas y valores finitos. La caché identifica el dataset y el codificador para impedir reutilizar características de otro modelo o archivo. El codificador preentrenado no se ajusta sobre UCI, así que la extracción puede realizarse una vez antes de las particiones.

5.1. Validación dentro de cada corrida

Se mantienen 30 particiones estratificadas 70 %/30 %, con semillas $2026, \ldots, 2055$. En cada una:

1. Se reserva el 30 % para prueba y no se usa al seleccionar parámetros.
2. Se divide el entrenamiento en 80 % de ajuste y 20 % de validación.
3. Se prueban tasas $\{0.1, 0.5, 1, 2, 5, 10, 20, 50\}$ y épocas $\{250, 500, 1000, 2000, 4000\}$. Se elige mayor F1 de validación y, en caso de empate, menor pérdida de validación.
4. Se aplica automáticamente la combinación ganadora al entrenar un modelo nuevo sobre todo el 70 % de entrenamiento; se evalúa una vez en su prueba.

Antes se calibraba una sola vez con la primera partición y se reutilizaba la elección en las otras 29. Esa validación incluía 244 mensajes que luego estaban en la prueba de la semilla 2027 y 226 de la semilla 2028. Por tanto esas pruebas no estaban aisladas de la selección global. No se puede afirmar que el sesgo fuera pequeño sin medirlo. Además, la calibración solo imprimía una advertencia si su ganador no coincidía con las constantes manuales; ahora se consume el ganador directamente.

5.2. Resultados de la nueva ejecución

Se ejecutaron las 30 corridas completas con MPNet/768, antes de ejecutar la demostración 1.3: la evaluación no necesita sus datos ni su entrenamiento. Las cifras son medias y desviaciones muestrales entre corridas.

Métrica	Media	Desviación	Mínimo	Máximo
Tasa de aciertos	0.9906	0.0016	0.9874	0.9940
F1	0.9641	0.0063	0.9519	0.9776

En la primera corrida la validación eligió tasa 20 y 4000 épocas, con F1 0.9714. Es un ejemplo de selección interna, no una constante para las demás corridas: sus elecciones se registran en la tabla exportada de hiperparámetros. La mejora frente a cifras históricas no permite atribuir el cambio solo al codificador, porque también se corrigió el protocolo de selección.

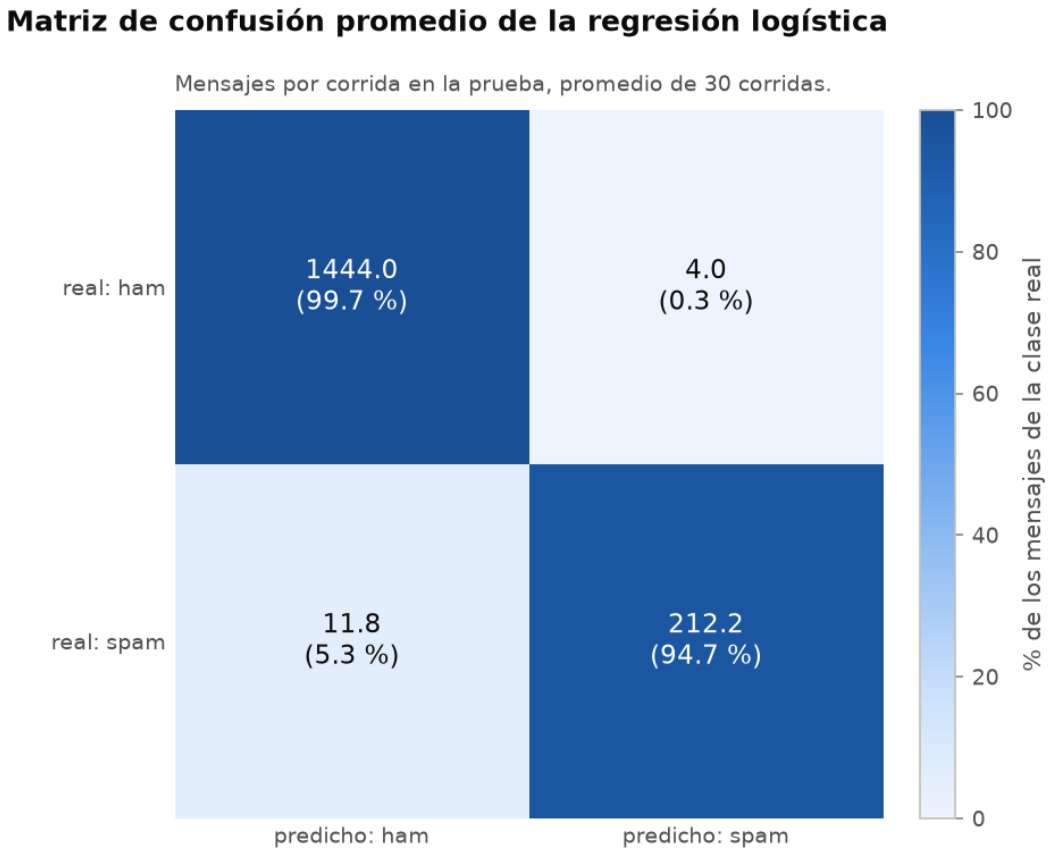


Figura 1: Errores medios de las 30 pruebas con MPNet/768, después de la calibración interna de cada corrida.

6. Punto 1.5: reducción y comparación con PCA

Con $D = 768$ se comparan $k = 384$ (50 %), $k = 192$ (25 %) y $k = 2$ con el caso sin PCA. Se conserva PCA propio por SVD de datos centrados, cálculo en doble precisión y signo fijo de cada componente. La proyección recibe la media y componentes obtenidas del conjunto de ajuste correspondiente.

La calibración se repite para cada corrida y cada k. Durante la selección, el PCA se ajusta solo con el ajuste interno y transforma la validación. Tras elegir tasa y épocas se reajusta PCA sobre todo el entrenamiento externo; se entrena allí y se transforma la prueba sin reajustarlo. Comparar las mismas semillas permite calcular diferencias pareadas de F1. Las proyecciones y la varianza de todo el dataset son diagnósticos descriptivos, no criterios para consultar la prueba al elegir hiperparámetros.

6.1. Resultados y dimensión óptima

Se completaron las 90 corridas con PCA y las 30 sin PCA, con las mismas semillas. La varianza corresponde al PCA descriptivo sobre todo el dataset; las métricas usan únicamente PCA ajustados con el entrenamiento correspondiente.

k	Varianza	Tasa de aciertos	F1	Δ F1 pareado
768	100 %	0.9906 ± 0.0016	0.9641 ± 0.0063	—
384	96.16 %	0.9905 ± 0.0019	0.9637 ± 0.0075	-0.0004
192	84.13 %	0.9894 ± 0.0021	0.9596 ± 0.0082	-0.0045
2	8.08 %	0.9466 ± 0.0054	0.7845 ± 0.0189	-0.1796

Con 384 componentes el F1 es menor que sin PCA en 14 de las 30 particiones; la diferencia pareada es -0.0004 ± 0.0030. Con 192, baja en 24 de las 30, con diferencia -0.0045 ± 0.0054. Con 2 baja en todas, con diferencia -0.1796 ± 0.0198: se escapan 61.4 spam por prueba frente a 11.8 sin PCA, y los ham marcados como spam suben de 4.0 a 27.9.

Las 384 componentes conservan casi toda la varianza y un F1 medio próximo al del caso completo. Las 192 vuelven a reducir a la mitad el número de características con mayor pérdida media. Dos dimensiones sirven para visualizar, pero pierden información relevante para clasificar. La desviación entre particiones solapadas no es un intervalo de confianza de muestras independientes. No se atribuye significación estadística a la diferencia pequeña con 384 componentes.

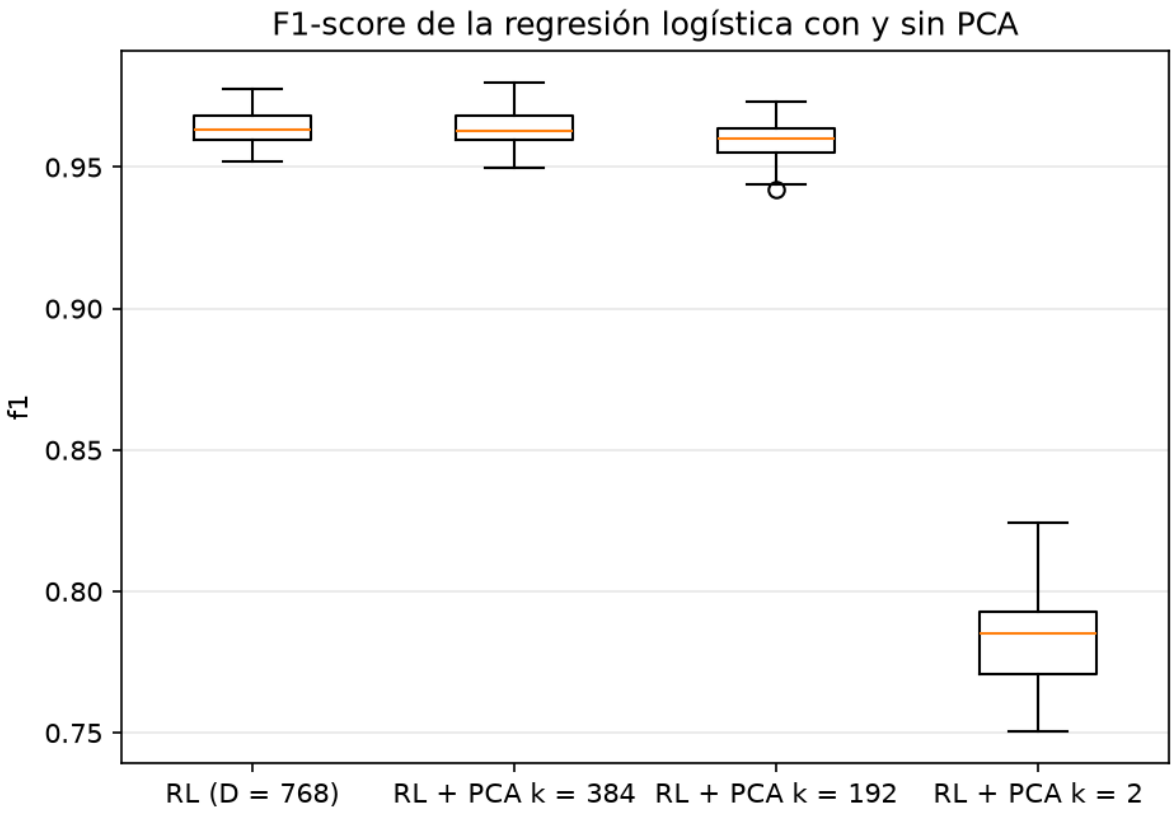


Figura 2: F1 en las mismas 30 particiones, con validación interna por método y corrida.

El equipo confirmó `OPTIMAL_PCA_DIMENSIONS = 384` el 24 de octubre de 2026. Se reduce a la mitad la dimensión inicial de MPNet y se conserva el 96.16 % de la varianza descriptiva. El F1 medio es 0.9637 frente a 0.9641 sin PCA, con diferencia media pareada de -0.0004. Esta cercanía no demuestra equivalencia estadística; se adopta como compromiso entre reducción y clasificación observada para la regresión logística. La decisión se basa en MPNet/768 y no en las recomendaciones anteriores de MiniLM/384. El detector del punto 2 compara estos mismos casos con su propia validación (Sección 7).

7. Punto 2.1: densidades por histograma y umbral

Lógica. En cada partición se ajusta un PCA solo con el entrenamiento. Por componente se estima un histograma de 30 intervalos de igual ancho entre el mínimo y el máximo de los datos, normalizado para que integre 1. El puntaje es $\sum_d \log p_d(x_d)$; un valor en un intervalo vacío o fuera del soporte recibe $\varepsilon = 10^{-12}$ ($\log \varepsilon \approx -27.6$) sin recortar el dato. Un puntaje menor que el umbral marca spam.

Pasos.

1. Se implementaron `fit_histogram_densities`, `per_dimension_log_likelihood`, `log_likelihood`, `select_threshold` (cuantil con interpolación lineal) y las gráficas, hasta superar las nueve pruebas ya escritas para el 2.1.

2. `fit_anomaly_detector` reparte los ham del entrenamiento: el 80 % construye los histogramas y el 20 % reservado da los puntajes del cuantil 5 %. Si el umbral sale de los mismos ham

de los histogramas, ninguno cae fuera del soporte y el umbral queda sesgado. En la primera partición, con 384 componentes, quedaba marcado el 17.3 % de los ham de prueba en lugar del 5 % (7.9 % con ham reservado); con 2 componentes el efecto es menor: 5.9 % frente a 4.4 %.

3. Con 384 componentes el detector casi no encuentra spam. Se diagnosticó con la brecha por componente (media de $\log p_d$ de ham menos la de spam) y se compararon los casos de 1.5 en la validación interna de los 30 entrenamientos, sin usar la prueba de esa misma corrida.

4. Se agregaron pruebas de suma por dimensión, umbral con filas reservadas, candidatos con columnas iniciales y solo ham, signo de la brecha y títulos de las figuras nuevas; `tests/test_punto2.py` reúne 15 pruebas en total.

Resultados de la selección. Validación interna (20 % estratificado del entrenamiento, como en 1.4) de las 30 particiones; media ± desviación:

D	Tasa de aciertos	F1	Spam detectado	Ham marcado	Gana
384	0.7812 ± 0.0141	0.0401 ± 0.0193	3.4 %	10.3 %	0
192	0.8425 ± 0.0072	0.0237 ± 0.0191	1.4 %	2.9 %	0
2	0.9201 ± 0.0106	0.7085 ± 0.0318	72.1 %	4.9 %	30

En la primera partición, la brecha de la componente 1 es 12.8 y es negativa en 330 de las 384 componentes: el spam está más concentrado que el ham y recibe más densidad. La brecha acumulada vale +12.6 con 2 componentes, cruza cero en 169, vale −2.8 con 192 y −41.1 con 384. Su máximo (14.2, con 11) no se evaluó porque el equipo restringió la búsqueda a los tres casos de 1.5. El enunciado pide reutilizar la D óptima anterior, sin prescribir esa búsqueda para el detector.

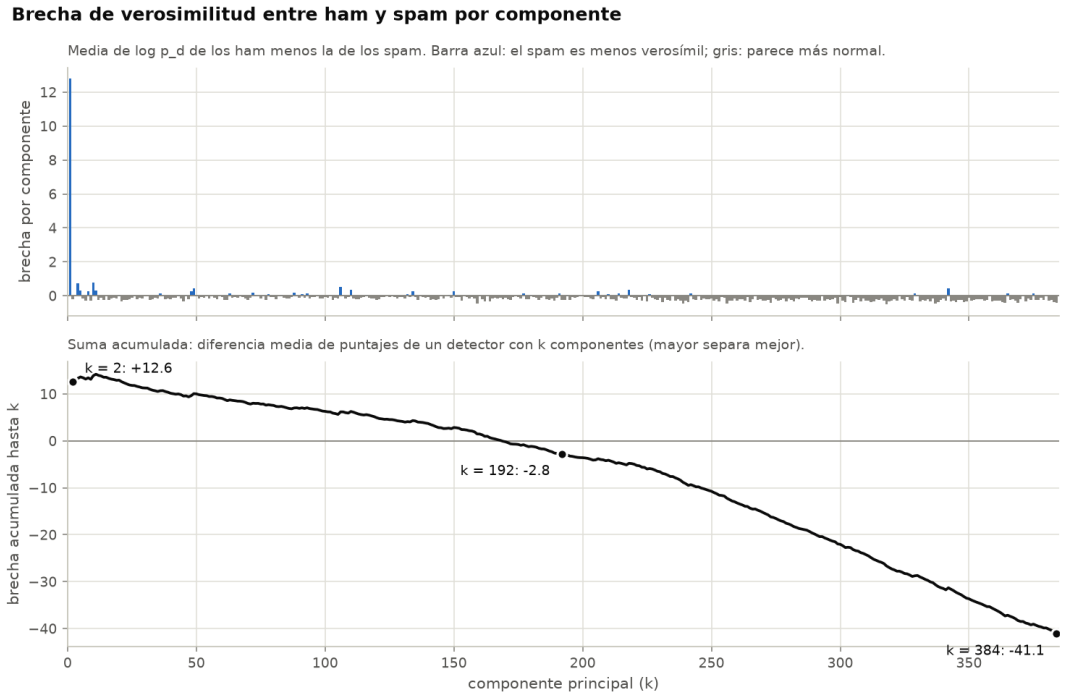


Figura 3: Brecha de verosimilitud por componente y acumulada (primera partición, validación).

Demostración con $D = 2$. Semilla 2026: 2703 ham construyen los histogramas, 676 fijan el umbral $\tau = -0.8334$. En su prueba queda bajo el umbral el 4.4 % de los ham y el 75.9 % del spam (tasa de aciertos 0.9294, F1 0.7424). El 50 % del spam queda fuera del soporte de la componente 1 y el 25.9 % cae en zonas de baja densidad; el resto se mezcla con el ham.

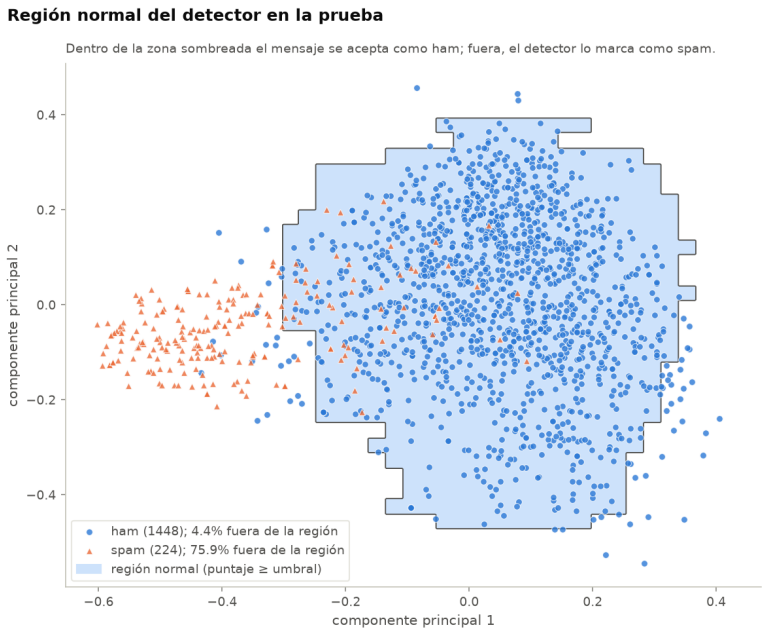


Figura 4: Región normal del detector con $D = 2$ y mensajes de prueba (semilla 2026).

8. Punto 2.2: evaluación en 30 corridas

Pasos. La celda del 2.2 repite el detector del 2.1 en las 30 particiones de la regresión (semillas 2026–2055), con PCA, histogramas y umbral ajustados solo con cada entrenamiento. Además de la tabla de medias, compara corrida a corrida con la regresión sin PCA y con la regresión de $k = 2$, y reporta los errores por clase. Para tener los resultados por partición del punto 1 se volvieron a ejecutar sus 120 corridas en esta máquina: las tablas de métricas coinciden exactamente con las versionadas.

Resultados. Media ± desviación; errores promedio por prueba (1448 ham y 224 spam):

Método	Tasa de aciertos	F1	Ham marcado	Spam detectado
Regresión, 768	0.9906 ± 0.0016	0.9641 ± 0.0063	4.0	94.7 %
Regresión, $k = 2$	0.9466 ± 0.0054	0.7845 ± 0.0189	27.9	72.6 %
Histogramas, $D = 2$	0.9195 ± 0.0084	0.7043 ± 0.0247	70.5	71.4 %

Diferencia pareada de F1 del detector: -0.2597 ± 0.0253 frente a la regresión sin PCA y -0.0802 ± 0.0212 frente a $k = 2$; menor en las 30 corridas en ambos casos. El F1 de validación del 2.1 (0.7085) anticipó el de prueba (0.7043). Con 2 componentes, ambos métodos detectan una fracción similar de spam; el detector marca más ham (4.9 % contra 1.9 %) porque su umbral acepta el 5 % de los ham y no sabe hacia dónde está el spam.

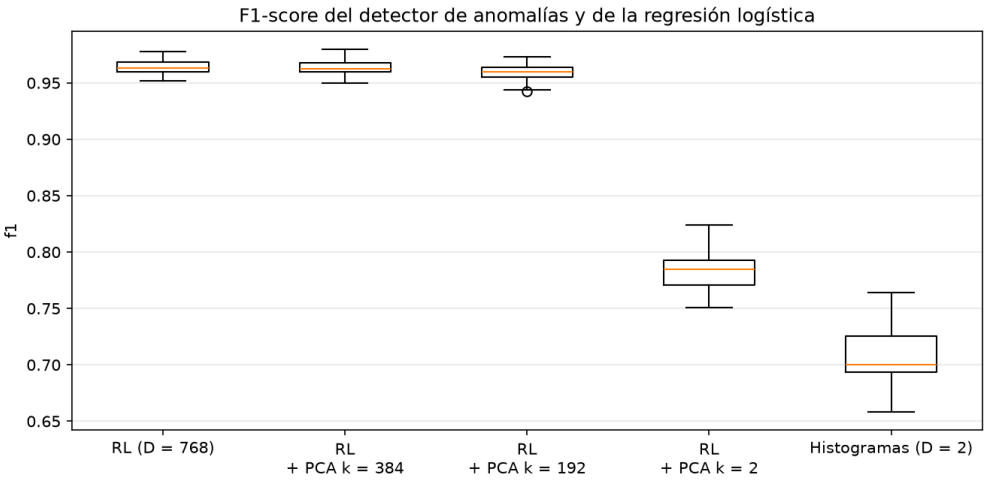


Figura 5: F1 del detector y de la regresión en las mismas 30 particiones.

9. Auditoría de los puntos 1 y 2 (24/10/2026)

9.1. Cobertura del enunciado

Se revisaron las definiciones y llamadas del cuaderno, las pruebas externas, las tablas exportadas y las secciones LaTeX contra las dos páginas del enunciado. El punto 3 queda fuera del alcance de esta auditoría.

1.1 y 1.2. Implementaciones propias de evaluación y descenso del gradiente en PyTorch, sin capas ni optimizadores prefabricados. Hay dos pruebas obligatorias por función. Una prueba adicional descubrió que `abs` anulaba la derivada de la sigmoide en logit cero; la selección explícita de la rama conserva $\sigma'(0) = 1/4$. Los valores del forward y el gradiente manual del entrenamiento se conservan.

1.3. Datos sintéticos separables, con prueba independiente, tablas, pérdida y fronteras. El ángulo casi coincide con el verdadero, pero el sesgo normalizado sigue desplazado: clasificar todo bien no identifica una única frontera verdadera.

1.4 y 1.5. Codificador MPNet, 30 particiones 70/30, tasa de aciertos, F1 de spam y PCA a 384, 192 y 2 componentes. La selección de tasa y épocas y el ajuste de PCA excluyen la prueba de cada corrida. El PCA sobre todos los datos se usa solo para figuras descriptivas. Las secciones del informe ya estaban redactadas; se corrigen sus afirmaciones y se amplía la evidencia.

2.1. Histogramas normalizados por dimensión, suma de logaritmos, umbral y figuras implementados y probados. La decisión del equipo de usar $D = 2$ difiere de reutilizar $D = 384$, como pide la lectura literal. Se conserva esa decisión y se añade una referencia con 384 componentes; queda pendiente validar con el docente si acepta la variante.

2.2. Se evalúan 30 particiones compartidas con el punto 1 y se comparan ambas métricas y los errores por clase. No debe darse por cerrado el cumplimiento de 2.1 únicamente por el mejor F1 de la variante.

9.2. Límites de las conclusiones anteriores

Las 30 particiones se solapan. Cada validación interna excluye su propia prueba, pero agregar todas para elegir una única dimensión permite que una fila de prueba de una corrida participe en la decisión a través de otra. Aquí $D = 2$ gana en las 30 validaciones: seleccionarla por corrida da la misma dimensión observada, sin demostrar ausencia de sesgo de desarrollo. La elección del compromiso $D = 384$ en 1.5 también se apoyó en la comparación de métricas de prueba; sus resultados se interpretan como comparación descriptiva y no como evaluación independiente de una dimensión seleccionada. La cercanía entre F1 de validación y prueba no prueba ausencia de sesgo. La selección de dimensión por F1 usa etiquetas de spam: las densidades son de una clase, pero todo el procedimiento no es enteramente no supervisado. El cuantil de ham reservado fija una fracción objetivo en calibración y no garantiza esa tasa en datos futuros. El número total de ham tampoco garantiza que cada intervalo de las colas esté bien poblado.

9.3. Experimento adicional reproducible

La celda opcional posterior al punto 2 conserva las constantes vigentes. Compara la cuadrícula de regresión con su ampliación a tasa 100 y 8000 épocas. Para el detector compara $D = 384$, $D = 2$ vigente y una selección interna de $D \in \{384, 192, 2\}$, intervalos $\{15, 30, 60\}$ y cuantiles $\{0.01, 0.025, 0.05, 0.1\}$. Cada alternativa se selecciona exclusivamente con el entrenamiento de su corrida. También cuenta textos idénticos compartidos por entrenamiento y prueba y calcula un F1 diagnóstico sobre textos no vistos. Ese subconjunto no equivale a una partición agrupada ni cuantifica por sí solo el sesgo por duplicados. Las alternativas son exploratorias: se diseñaron después de conocer los resultados previos del mismo dataset y no sustituyen una evaluación nueva. La persona autorizó estas corridas con la caché local y sin descargas.

Resultados de las 30 corridas por variante.

Variante	Tasa de aciertos	F1
Regresión vigente	0.9906 ± 0.0016	0.9641 ± 0.0063
Regresión, cuadrícula ampliada	0.9904 ± 0.0018	0.9636 ± 0.0068
Histogramas, $D = 384$	0.7892 ± 0.0119	0.0386 ± 0.0145
Histogramas, $D = 2$ vigente	0.9195 ± 0.0084	0.7043 ± 0.0247
Histogramas, calibración por corrida	0.9321 ± 0.0061	0.7173 ± 0.0441

La ampliación de la regresión mejora F1 en 2 corridas, empata en 22 y empeora en 6: diferencia pareada -0.0005 ± 0.0023. Se conserva la cuadrícula vigente. El detector calibrado mejora en 24 y empeora en 6, con diferencia $+0.0130 \pm 0.0519$. Elige siempre $D = 2$, cuantil 1 % o 2.5 % en 27 corridas y 5 % en 3. Reduce los ham marcados de 70.5 a 35.4 por prueba, pero aumenta el spam escapado de 64.0 a 78.1 y la desviación de F1 de 0.0247 a 0.0441. Su F1 medio de validación es 0.7480 y el de prueba 0.7173; no se adopta automáticamente esa alternativa.

Entre 145 y 181 textos de prueba por corrida ya aparecen en entrenamiento; media 162.2 (9.70 %). El F1 de la regresión vigente sobre textos no vistos es 0.9565 ± 0.0076. Esto no cuantifica el sesgo por duplicados: es un subconjunto distinto y no una partición agrupada. Las tres tablas completas, versionadas en la rama de auditoría, se exportan desde la celda opcional a `results/tables/auditoria-*.csv`; la implementación permanece en el notebook, sin copias Python. La celda queda desactivada por defecto; se ejecutó con activación temporal y autorización para estas corridas.

Selección mínima para main (25/10/2026). La rama de auditoría conserva la evidencia experimental completa. En main se incorporan las correcciones del notebook, las pruebas, el ejecutor portátil con sus dependencias, las fuentes LaTeX y ambos PDF para continuar el punto 3. Los CSV experimentales adicionales y los PNG regenerados quedan en la rama de auditoría; main conserva sus figuras anteriores y recompila los PDF con ellas. No se adoptan las alternativas de calibración ni se cambian las constantes.

Paquete para el equipo (25/10/2026). La persona confirmó conservar las configuraciones vigentes y mantener las sugerencias documentadas. Se incorpora `scripts/run_puntos_1_2.py`: selecciona las celdas existentes, valida la caché antes de limpiar resultados y usa un kernel nuevo del intérprete invocante. No duplica la implementación del notebook. `-check` no ejecuta experimentos; `-audit` activa las alternativas solo en una copia y `-allow-download` permite regenerar MPNet explícitamente. Se guarda un respaldo local de resultados y el notebook ejecutado en `build/`; el notebook canónico del commit queda sin salidas de desarrollo. La rama de auditoría conserva los CSV y PNG regenerados; main incorpora los fuentes, pruebas, dependencias, PDF y tabla de pruebas. La guía `docs/reproducibility.md` y `requirements-audit.txt` documentan la instalación, las versiones directas observadas y la regeneración de la caché; no se versionan entornos, cachés, pesos de modelos ni archivos privados de la máquina. Tres pruebas nuevas verifican selección, identidad de caché sin modificar resultados y ejecución de una celda mínima con el intérprete correcto. La comprobación local del ejecutor no requirió repetir las corridas largas.

10. Punto 3.1: clasificación con un modelo de lenguaje (26/10/2026)

10.1. Decisiones

La persona eligió, el 26/10/2026, `Qwen/Qwen2.5-3B-Instruct` como único modelo de 3.1 (3.2 y 3.3 quedan para después), un prompt sin ejemplos en inglés (`DEFAULT_PROMPT`) aplicado con la plantilla de chat del modelo, decodificación voraz y la regla de respuestas ambiguas: si la respuesta no contiene `spam` ni `ham`, cuenta como ham. Las celdas que cargan el modelo se ejecutan en una GPU CUDA. Las pruebas iniciales del equipo se hicieron en CPU; la evaluación actual usa el servidor WSL con RTX 3080 Ti conectado a PyCharm mediante Quick Tunnel.

10.2. Lógica y pasos

1. `select_llm_device(require_gpu=True)` devuelve `cuda` o detiene la celda con instrucciones; nunca clasifica en CPU por omisión. `load_llm` carga tokenizador (relleno a la izquierda) y modelo en `bfloat16` (GPU con capacidad 8.0 o más) o `float16`, y registra la revisión descargada.

2. `generate_responses` arma los prompts con el texto original, agrupa los lotes por longitud para reducir relleno, genera hasta 8 tokens sin muestreo y decodifica solo los tokens nuevos, en el orden original.

3. `parse_label` toma la primera palabra `spam` o `ham`; una negación (*not spam*, *non-spam*) cuenta como ham y una respuesta sin ninguna de las dos, también. `is_ambiguous_response` permite contarlas.

4. Cada mensaje se clasifica una sola vez por modelo. Las respuestas se guardan en `data/processed/llm/` con el hash de los textos, el modelo, el prompt y la cantidad

de tokens; una caché que no coincide se rechaza.

5. `llm_sample_metrics` evalúa 30 muestras estratificadas del 30 % (semillas 2026 a 2055) y agrega la matriz de confusión de cada muestra.

Las celdas del notebook separan selección del modelo, carga (GPU), prueba rápida (GPU), clasificación completa (GPU), liberación de memoria y evaluación. Cambiar de modelo solo requiere cambiar `LLM_MODEL_NAME` y repetir las cinco primeras.

10.3. Pruebas y verificación

Se implementó la prueba recomendada de `parse_label` y siete más: negaciones y ambigüedad, exigencia de GPU, orden de las respuestas con un tokenizador y un modelo simulados, contrato de `classify_messages`, validación de la caché, muestreo de las métricas y resumen de respuestas. También se comprobó `generate_responses` con un Qwen2 diminuto de pesos aleatorios creado localmente, sin descargas. Se agregan dos regresiones: revisión compartida por tokenizador y pesos sin depender de `config._commit_hash`, y ejecución de pruebas con fixtures pytest. El ejecutor anterior llamaba las funciones sin argumentos: provocaba errores con `monkeypatch` y `tmp_path`. Ahora usa pytest, importación por `importlib` y recarga de los módulos. La suite externa y la celda del notebook en WSL dan 58/0/0. Se sincronizó también el archivo guardado remoto: ejecutar el código del editor no actualizaba automáticamente el archivo que lee el cargador de pruebas.

10.4. Resultado de la línea base

Con revisión `aa8e72537993ba99e69dfaafa59ed015b17504d1`, `bfloat16`, lote 16 y PyTorch `2.14.0+cu130`, se generaron 5574 respuestas en 84.2 segundos. Los pesos ya estaban en caché; el tiempo excluye descarga y carga. Se reprodujo exactamente la caché anterior y se liberó el modelo de la GPU después de generar las respuestas.

En 30 muestras estratificadas del 30 %, semillas 2026 a 2055, se obtuvo tasa de aciertos 0.7853 ± 0.0073 y F1 0.5452 ± 0.0090. No hubo respuestas ambiguas. En todo el dataset hubo 717 verdaderos positivos, 1168 falsos positivos, 30 falsos negativos y 3659 verdaderos negativos. La sensibilidad para spam fue 96.0 %, pero su precisión fue 38.0 %: el exceso de ham marcado como spam explica el F1 bajo. Por muestra hubo, en promedio, 349.9 falsos positivos y 9.0 falsos negativos.

La regresión sin PCA conserva F1 0.9641 ± 0.0063 y el detector vigente, 0.7043 ± 0.0247. Son referencias descriptivas: sus pruebas de particiones 70/30 no son las mismas filas que las muestras de 3.1, por lo que no se afirma una comparación pareada. El prompt y la regla de interpretación se mantienen; no se ajustaron después de observar estos resultados. Se exportan las 30 filas, resúmenes, detalles de ejecución y dos figuras. El análisis también se incorpora al notebook y a la sección 3 del informe.

10.5. Cierre de verificación

La evaluación se repitió desde la caché mediante las celdas de PyCharm, sin cargar el modelo ni repetir inferencia. Las 30 muestras reprodujeron las métricas y la celda de pruebas volvió a dar 58/0/0. Se verificó la igualdad de las fuentes de las 65 celdas guardadas en WSL con las locales. Los subtítulos de las dos figuras ahora describen muestras del 30 %; las figuras y las seis tablas se transfirieron comprobando SHA-256. Se conserva un respaldo ejecutado antes de limpiar las salidas del notebook de desarrollo. La configuración y los algoritmos de los puntos 1 y 2 no cambian. Se recompilan ambos PDF con las figuras corregidas. Este cierre se publicó después en `2534503` por autorización expresa.

11. Punto 3.2: modelos adicionales cuantizados (26/10/2026)

El cierre de 3.1 se respaldó en origin/main mediante el commit `2534503`. Después la persona confirmó `mistralai/Mistral-7B-Instruct-v0.3` y `Qwen/Qwen2.5-14B-Instruct`, ambos con NF4 de 4 bits, doble cuantización y cálculo bfloat16. Se autorizó su descarga y evaluación secuencial en WSL. El prompt, la generación voraz de hasta 8 tokens, la interpretación de respuestas y las 30 muestras de 3.1 se conservan.

Se agrega un cargador separado, sin cambiar la firma de `load_llm`. Fija revisión de tokenizador y pesos, exige CUDA y mantiene todo el modelo en una GPU. La caché NF4 incorpora revisión, cuantización, lote y versiones; la caché original de 3.1 conserva su identidad. Las seis pruebas nuevas protegen la carga cuantizada y evitan confundir esas cachés: la suite local da 64 superadas. En WSL se instalaron `accelerate==1.15.0` y `bitsandbytes==0.50.2`, sin modificar PyTorch ni Transformers. Las fuentes y alternativas se conservan en `docs/research/punto3-2-modelos-cuantizados.md`.

11.1. Resultados y verificación de 3.2

Se completaron los 5574 mensajes por modelo y las mismas 30 muestras estratificadas del 30 %, sin modificar el prompt ni la interpretación.

Configuración	Tasa de aciertos	F1-score
Qwen 3B, BF16	0.7853 ± 0.0073	0.5452 ± 0.0090
Mistral 7B, NF4	0.7923 ± 0.0084	0.5518 ± 0.0108
Qwen 14B, NF4	0.8954 ± 0.0065	0.7126 ± 0.0139

Tabla 1: Media y desviación muestral en las mismas 30 muestras.

Qwen 14B reduce los falsos positivos de 1168 a 552 frente a Qwen 3B y los falsos negativos de 30 a 26. Mistral tiene 1122 falsos positivos y 35 falsos negativos. Sus doce respuestas ambiguas son ham; Qwen 14B no tiene respuestas ambiguas. La diferencia pareada media de F1 es +0.0066 para Mistral menos Qwen 3B (23/30 positivas), +0.1675 para Qwen 14B menos Qwen 3B y +0.1609 para Qwen 14B menos Mistral (30/30 positivas en ambos). Son comparaciones descriptivas con muestras solapadas, sin prueba de significación ni atribución causal al tamaño o a cuantizar.

Mistral usó lote 16, 168.6 s y picos de 4.625 GiB asignados y 5.078 GiB reservados por PyTorch; Qwen 14B, lote 4, 536.8 s y picos 9.774/9.984 GiB. Los tiempos excluyen descarga y carga; la memoria no incluye otros procesos ni el controlador. Ambos modelos residieron completos en GPU y se liberaron al terminar.

Se exportaron ocho figuras y once CSV, junto con tres cachés excluidas de Git: 22 artefactos transferidos y verificados por SHA-256. Se miraron todas las figuras. En el Mac, las funciones del notebook reprodujeron las 30 filas de cada modelo desde sus cachés, con tolerancia 10^{-12} respecto de los CSV. La celda WSL dio 64/0/0, igual que pytest local. El notebook conserva una única implementación. Las fuentes de las 69 celdas guardadas coinciden entre Mac y WSL; ambas copias de desarrollo quedan sin salidas y se conserva un respaldo ejecutado. La persona autoriza el commit y la publicación de este estado de 3.2 en origin/main antes de iniciar 3.3; solicita consultar las decisiones de fine-tuning y mantener el alcance del enunciado y del curso.

11.2. Conexión local y recuperación de la carga

Jupyter y el reenvío de Windows solo escuchaban en localhost. Se publicó TCP 8888 en la dirección LAN de Windows mediante `portproxy`, conservando el servidor existente. El firewall permite únicamente la dirección del Mac en la interfaz LAN; no se cambió el perfil de red. PyCharm mostró conexión correcta y ejecutó una celda por MCP en Linux con CUDA y la RTX 3080 Ti. Sin autenticación, la API devuelve HTTP 403; la página de acceso devuelve HTTP 200. En tres consultas por ruta, la mediana fue 0.035 s por LAN y 0.374 s por el túnel: es una comprobación breve de HTTP, no una medición de inferencia. La configuración y reversión están en `docs/jupyter-lan.md`. Se retiró únicamente el túnel de Jupyter después de verificar la LAN; la dirección pública dejó de responder y la conexión directa siguió activa.

El kernel antiguo ya no existía. Su salida conservada registra un error de ZeroMQ de archivos abiertos durante actualizaciones de widgets Xet; no basta para identificar por sí solo el primer recurso agotado. Los tres archivos de pesos de Mistral, 14 496 080 928 bytes, estaban completos, verificados contra los SHA-256 oficiales y protegidos con enlaces duros. Se recuperaron sin repetir la descarga. Las cargas siguientes usan HTTP y desactivan las barras de Hugging Face. Mistral cargó en NF4: 3.75 GiB de pesos y buffers; la prueba rápida pasó con el prompt máximo de 307 tokens y lote 16. La red se configuró sin reiniciar Jupyter ni WSL.

12. Punto 3.3: ajuste eficiente

El cierre de 3.2 se publicó en origin/main mediante `88b6ee9`. La persona confirmó Qwen 14B con LoRA sobre NF4, PEFT y un bucle explícito de PyTorch. Se hará un entrenamiento con partición externa 70/30 fija; del entrenamiento se reserva 20 % para validar. Se aprobaron rango 8, alpha 16, tasa 2×10^{-4}, una época, lote físico 1 y acumulación de 16; también opciones adicionales de revisión y configuración en la firma, conservando sus argumentos originales. Solo se supervisan la respuesta ham/spam y EOS; la prueba queda fuera del ajuste.

La base y los adaptadores cargaron con 34 406 400 parámetros entrenables y 9.18 GiB de pesos y buffers. La prueba de capacidad pasó con un ejemplo de 314 tokens, dos pasos de AdamW y dos microbatches por paso; alcanzó 10.1925 GiB asignados y 10.3359 GiB reservados. Adaptadores y aleatoriedad se restauraron. La partición tiene 3121 filas de ajuste, 781 de validación y 1672 de prueba; 127 textos de prueba aparecen en ajuste (150 en el entrenamiento externo completo). Se mantiene la partición por filas y se informa esta limitación; no se deduplica ni se altera el dataset.

PEFT 0.21.0 se instaló sin modificar las demás dependencias. La base permanece congelada en NF4/BF16 y solo se entrenan matrices LoRA. Se usa gradient checkpointing y pérdida en FP32; conservar las matrices base BF16 evita la promoción completa a FP32 del preparador genérico de PEFT, que aumenta la memoria. La capacidad se comprobó con backward, acumulación y AdamW, restaurando pesos y aleatoriedad antes del ajuste. Las pruebas locales de máscaras, pérdida causal, gradientes, partición y restauración, exportación de figura y procedencia de cachés dan 77 superadas.

12.1. Ejecución y resultados de 3.3

La época completa se ejecutó en la celda de ajuste sin interrupciones: 196 pasos de AdamW, 3498.5 s de ajuste y validación final, picos de 10.19 GiB asignados y 10.37 GiB reservados. La pérdida de validación bajó de 0.5163 a 0.0446. Se guardaron adaptador, historial y checkpoints cada 50 pasos en `data/processed/lora/qwen14b-dda32081338adef6/` de WSL. Después se clasificaron las 1672 filas de prueba en 224.2 s (lote 4, pico de 9.81 GiB asignados) y se liberó el modelo de la GPU.

Tabla 2: Qwen 14B NF4 antes y después de LoRA en la prueba fija.

Modelo	Tasa de aciertos	F1	VP	FP	FN	VN
Sin ajuste	0.8935	0.7120	220	174	4	1274
Con LoRA	0.9922	0.9708	216	5	8	1443

Los falsos positivos bajan de 174 a 5 y los falsos negativos suben de 4 a 8. En las filas emparejadas, el ajuste corrige 172 errores de la base e introduce 7. En las 1545 filas sin texto presente en ajuste, F1 pasa de 0.7024 a 0.9688; solo 1 de los 13 errores del modelo ajustado está en una fila repetida. No hubo respuestas ambiguas. Al ser un único entrenamiento, la comparación con las 30 corridas de los puntos 1 y 2 es descriptiva.

Verificación. Los 14 artefactos pequeños (cuatro JSON, siete CSV y tres PNG) se transfirieron a Mac con SHA-256; adaptador, checkpoints y pesos se quedan en WSL. En CPU se reprodujo el identificador del experimento con las funciones del notebook y `load_sms_spam`; se validaron solicitud, hashes del adaptador e historial, y se recalcularon las métricas (diferencia máxima 1.1×10^{-16}). Las tres figuras se revisaron visualmente. Después, `RUN_LORA_TRAINING` quedó en falso y la celda de ajuste reutiliza el adaptador guardado si faltan respuestas. Las celdas cambiadas y las pruebas nuevas se sincronizaron con WSL, con respaldo previo de sus archivos, y su celda de pruebas dio 77/0/0.

13. Puntos pendientes

El punto 2 está implementado; su conformidad literal requiere resolver la variante de dimensión. Lo que falta, en orden (detalle en `TODO.md`):

1. Validar con el docente si acepta $D = 2$ como variante del apartado 2.1. La referencia con $D = 384$ ya tiene 30 corridas; la persona confirmó mantener la configuración vigente y no adoptar las alternativas.

2. Avisar al equipo que el detector vigente usa 2 componentes y la regresión 384; las constantes no se cambiaron durante la auditoría.

3. Puntos 3.1, 3.2 y 3.3 completos y publicados en origin/main; el cierre de 3.3 se publicó por solicitud expresa de la persona.

4. Completar la comparación de métodos y las conclusiones del informe.

5. Confirmar integrantes y fecha de entrega; después, ejecución final completa, Doxygen, PDF final y `.zip`.

14. Continuación en main y kernel GPU remoto (26/10/2026)

La persona eligió Cloudflare Quick Tunnel sin cuenta ni dominio y conectó PyCharm a Jupyter mediante su token. El equipo comparte el host de a una persona o un trabajo GPU a la vez, sin kernels dedicados garantizados. El esquema SSH/Tailscale no es requisito; sus fuentes permanecen en `codex/wsl-gpu-host`, sin integrarlas a main.

Los clones del Mac y de WSL volvieron a main, base `d520690`, mediante cambios de rama sin descartar archivos. Se comprobó que los notebooks conservaran su hash antes y después. La celda agregada por la persona en el Mac y los archivos auxiliares no versionados de PyCharm en WSL se conservaron. No se reinició ni se cerró el kernel remoto.

Una consulta breve por las herramientas del notebook confirmó Linux/WSL2, el entorno virtual del proyecto, PyTorch `2.14.0+cu130`, CUDA disponible y la GPU NVIDIA GeForce RTX 3080 Ti. Esta comprobación verifica identidad y disponibilidad de CUDA, no una inferencia ni un entrenamiento. Las pruebas locales de main dan 48 superadas, una pendiente del punto 3 y cero fallidas; las tres pruebas de infraestructura de la otra rama no pertenecen a main.

Después se incorpora `db5db2e`, con Qwen2.5-3B-Instruct, prompt y tratamiento de respuestas ya confirmados. Se descarta solo la celda local de diagnóstico CUDA, con respaldo, y se conservan las implementaciones y pruebas del equipo. Los conflictos en este registro y en las instrucciones se reconcilian; el PDF se regenera desde el TeX. Tras incorporar el commit había 56/0/0; después de las dos regresiones nuevas hay 58/0/0. La persona autoriza descargar y evaluar 3.1 en WSL. En ese momento los modelos adicionales y LoRA/DoRA estaban pendientes; 3.2 se confirmó y evaluó después, como se documenta arriba. No cambian las constantes de 1 y 2. Las órdenes de terminal del Mac siguen siendo locales; para trabajar con la GPU se usa el kernel remoto conectado. No se guardan URL ni tokens del túnel en el repositorio.

La persona confirma el login interactivo de Hugging Face en WSL. Se usa un token dedicado de lectura y alcance limitado, almacenado fuera del repositorio; la comprobación compartida solo informa si está presente, nunca su valor. Quienes compartan el usuario Linux pueden usar esas credenciales. La autenticación evita los límites anónimos de acceso/descarga, pero no acelera la inferencia local. La suspensión accidental ya interrumpió la disponibilidad: preservar el estado exige mantener Windows, WSL y Jupyter vivos.

15. Registro de decisiones

Fecha	Decisión	Motivo
17/10/2026	Sigmoide estable y gradiente exacto propio.	Implementación transparente sin desbordamientos ni optimizadores externos.
19/10/2026	Frontera verdadera fija en 1.3.	Comparar la recta aprendida con una referencia con sesgo.
23/10/2026	1.3 independiente de UCI; dataset original versionado.	Evitar mezclar demostración sintética y evaluación real; compartir los mismos datos.
23/10/2026	PCA por SVD centrada y comparación pareada.	Ajuste propio y comparación sobre las mismas particiones.
24/10/2026	Único notebook con toda la implementación.	Sustituye archivos por punto y utilidades separadas; pruebas consumen las mismas definiciones.
24/10/2026	Mantener logística $Xw + b$ seguida de sigmoide.	Confirmación del equipo; coincide con 1.1 y 1.2 del enunciado.
24/10/2026	MPNet con 768 características; descargar y ejecutar 1.4–1.5.	Confirmación del equipo; reemplaza MiniLM/384 y sus resultados.
24/10/2026	Calibración interna por corrida y cantidad de componentes.	Aislar cada prueba y usar el ganador automáticamente.
24/10/2026	Conservar duplicados y documentar su limitación.	Mantener el dataset original y el protocolo por filas sin introducir otra evaluación.
24/10/2026	Fijar $D = 384$ como óptimo de 1.5 (regresión); el detector usa luego $D = 2$.	Decisión del equipo: mitad de las características, 96.16 % de varianza descriptiva y F1 medio 0.9637 frente a 0.9641 sin PCA; no implica equivalencia estadística.
24/10/2026	Densidades del punto 2 solo con ham del entrenamiento.	Describir la distribución normal; el spam queda como lo atípico.
24/10/2026	Umbral: cuantil 5 % de un 20 % de ham reservado; 30 intervalos.	Con los ham de los histogramas el umbral queda sesgado hacia arriba (con 384 componentes marcaba 17.3 % de ham en vez de 5 %).
24/10/2026	Detector con $D = 2$; la regresión conserva 384.	Mayor F1 de validación interna en las 30 particiones entre los casos de 1.5; con 384 el spam resulta más verosímil que el ham (F1 0.04).
24/10/2026	2.2: comparar el detector por partición con la regresión sin PCA y con $k = 2$.	Separar el efecto de la dimensión del efecto del método, con las mismas semillas.
24/10/2026	Corregir la derivada de la sigmoide en cero y añadir una prueba.	Autograd daba 0 en vez de 1/4; se conserva el entrenamiento analítico.
24/10/2026	Auditar con caché local; conservar las configuraciones vigentes.	Autorización para 30 corridas por variante, sin descargas. Ampliar la regresión no mejoró F1; calibrar histogramas reduce falsas alarmas, pero aumenta el spam escapado.
24/10/2026	Registrar la diferencia entre $D = 384$ literal y $D = 2$ del equipo.	La mejor métrica no demuestra cumplimiento del enunciado; validar la variante con el docente.

Fecha	Decisión	Motivo
25/10/2026	Conservar las configuraciones del equipo y compartir la reproducción.	Confirmación de la persona; las alternativas siguen documentadas sin adoptarse. Se incluye un ejecutor portátil, dependencias directas y guía; se excluyen cachés, modelos y entornos locales.
25/10/2026	Publicar la auditoría completa y seleccionar solo lo necesario para main.	Solicitud de la persona: notebook, Python, pruebas, fuentes LaTeX, PDF y sus dependencias; conservar la configuración vigente y continuar el punto 3.
26/10/2026	3.1 con Qwen2.5-3B-Instruct, prompt sin ejemplos en inglés y decodificación voraz; respuestas ambiguas como ham.	Elección de la persona. Sin ejemplos del dataset en el prompt no hay fuga hacia la evaluación; contar lo ambiguo como ham penaliza el spam no detectado y se informa su cantidad.
26/10/2026	Celdas del modelo en GPU CUDA (Colab u otra máquina), separadas de la evaluación.	La GPU local es AMD y PyTorch local es solo CPU; la caché de respuestas permite evaluar sin GPU y cambiar de modelo sin tocar otras celdas.
26/10/2026	Continuar en main con Jupyter remoto por Quick Tunnel.	Elección de la persona: sin cuenta ni dominio. PyCharm/MCP confirma WSL2, CUDA y RTX 3080 Ti. El equipo comparte la GPU de a un trabajo a la vez; se integra 3.1 de `db5db2e` y se autoriza su evaluación en WSL.
26/10/2026	Completar solo la línea base 3.1, sin cambiar el prompt.	Confirmación de la persona. Qwen logra F1 0.5452, con falsas alarmas altas. Se fija la revisión de pesos/tokenizador y se corrige el ejecutor pytest; 58 pruebas pasan también en el notebook.
26/10/2026	Respaldar el cierre de 3.1 en origin/main antes de iniciar 3.2.	Solicitud expresa de la persona: se autoriza commit y push del estado verificado. Los dos modelos adicionales se confirmaron posteriormente; el ajuste fino sigue pendiente.
26/10/2026	Evaluar Mistral 7B y Qwen 14B con NF4 de 4 bits.	Elección de la persona después del respaldo de 3.1: dos familias, revisiones fijadas y ejecución secuencial en WSL; mismo prompt y muestreo.
26/10/2026	Conectar PyCharm por LAN y conservar Jupyter en localhost.	Reenvío de Windows y firewall limitado al Mac; ejecución CUDA verificada. Recuperar pesos de Mistral por SHA-256, sin repetir la descarga.
26/10/2026	Publicar el cierre verificado de 3.2 antes de iniciar 3.3.	Solicitud expresa de la persona: commit y push; consultar modelo, técnica y evaluación del ajuste eficiente antes de entrenar.
26/10/2026	Qwen 14B con LoRA sobre NF4; una partición fija 70/30.	Elección de la persona: PEFT y bucle explícito de PyTorch; rango 8, alpha 16, tasa 2×10^{-4}, una época, lote 1 y acumulación de 16; validación solo del entrenamiento. La capacidad física pasó.

Fecha	Decisión	Motivo
26/10/2026	Completar el piloto 3.3 y evaluarlo una sola vez en la prueba reservada.	F1 0.7120 → 0.9708 y FP 174 → 5. Sin reentrenar ni ajustar parámetros con la prueba; `RUN_LORA_TRAINING` desactivado y reutilización del adaptador guardado.
26/10/2026	Publicar el cierre verificado de 3.3 en origin/main.	Solicitud expresa de la persona: commit y push, como en 3.2.

16. Verificación y reproducción

- `pytest` (26/10/2026, con 3.1 a 3.3): 77 superadas, 0 pendientes y 0 fallidas, también en la celda WSL. Al cerrar 3.2 eran 64/0/0; al cerrar 3.1, 58/0/0; antes del punto 3.1, 48/1/0. La prueba de la derivada reprodujo el fallo antes de la corrección y pasa después. Las pruebas usan el backend `Agg` de Matplotlib.
- Notebook guardado validado con `nbformat`, IDs únicos y AST. Las definiciones se cargan sin descargas ni experimentos. Se conservan sesgo, gradiente manual y contratos existentes.
- Dataset original sin modificaciones y caché MPNet verificada por metadatos, hash de textos ordenados, forma $(5574, 768)$, tipo `float32` y finitud. La auditoría no descargó modelos.
- Ejecución desde un kernel nuevo de los puntos 1 y 2, auditoría adicional y pruebas, en 407 segundos. Se conservaron las salidas en una copia local ejecutada; el notebook del commit se limpia según las reglas de desarrollo; la celda opcional de auditoría quedó desactivada para futuras ejecuciones. La ejecución final completa se realizará al cerrar la entrega.
- Las métricas de regresión reproducen exactamente las tablas anteriores. Las métricas del detector vigente también coinciden; hay diferencias numéricas pequeñas en umbrales y en validación con $D = 384$ respecto de Linux (tasa de aciertos 0.7812 frente a 0.7810). Se registran los valores de esta ejecución sin atribuirlos a cambios de modelo.
- Las 18 figuras regeneradas e inspeccionadas se conservan en la rama de auditoría. Sus diagramas de caja usan dos líneas y medianas neutras; main conserva sus PNG previos. No se editaron figuras ni tablas experimentales a mano.
- Revisión e informe preliminar compilados mediante los dos comandos `./build_pdf.sh review` y `./build_pdf.sh`; revisión visual de las páginas renderizadas. Los puntos 3.1, 3.2 y 3.3 se ejecutaron y documentaron; la comparación global y las conclusiones finales permanecen pendientes.

Para reproducir, instalar `requirements.txt` y ejecutar las definiciones, configuración, datos y puntos 1 y 2 completos del cuaderno. La auditoría requiere activar su celda opcional; se debe autorizar esa ejecución larga. La configuración limpia las exportaciones: regenerar las figuras antes de compilar documentos que las usen.